LangConfig — 团队安装指南

LangConfig — 团队安装指南（中文版）

> 在全新机器上从 zip 包安装 LangConfig：PostgreSQL 安装、uv、解压、依赖同步、 数据库种子（含工作区配置）、LLM API Key、健康检查与启动。

---

0. 你会收到什么

`langconfig.zip`（约 31 MB）包含完整源码，不含密钥/构建产物（无 `.env`、`node_modules`、`.venv`、`data`、`backend/outputs`、git 历史）。数据库种子会 恢复一个**与机器无关的基线**：44 个深度智能体、28 个 LLM 模型、58 个工具、 19 个技能。**你的工作区路径和 API Key 在下面步骤中本地添加——zip 里不包含 任何机器相关的信息。**

前置依赖——Ubuntu（22.04 / 24.04）

全新 Ubuntu 服务器在开始以下步骤前需要一些基础工具：

```
sudo apt update && sudo apt install -y unzip git curl ca-certificates
```

---

1. PostgreSQL — 安装与配置

> ⚠️ **在启动 Postgres 之前务必先做这一步：** `docker compose` 在 compose 时会读取 `.env` 来决定宿主机端口。如果你在创建 `.env`（第 5 步）之前就启动 容器， `POSTGRES_PORT` 会回退为 `5432`，文档中的 `5433` 映射不会生效。 **推荐顺序：先创建 `.env`（第 5 步），再启动 Postgres（本步骤）**——或者内联 传参：`POSTGRES_PORT=5433 docker compose up -d postgres`。

选择**一种**模式，并记下三项凭据——整个应用（后端、Alembic、种子脚本、docker-compose、SQL 智能体）都从它们派生连接信息。

1 / 12

项目	默认值	使用位置
数据库名	`langconfig`	`.env` 中的 `POSTGRES_DB`
用户名	`langconfig`	`.env` 中的 `POSTGRES_USER`
密码	`langconfig_dev`	`.env` 中的 `POSTGRES_PASSWORD`
主机 / 端口	`localhost`：原生 5432，Docker 5433	`.env` 中的 `POSTGRES_PORT`

方案 A — Docker（最简单，团队推荐）

```
# 在解压后的目录中执行
docker compose up -d postgres
# 容器：langconfig-postgres，宿主机端口 5433（→ 容器 5432）
# 健康检查：docker ps   →  "langconfig-postgres  0.0.0.0:5433->5432/tcp"
```

> **Ubuntu 还没装 Docker?**
>
> ```
> curl -fsSL https://get.docker.com | sudo sh      # 安装 docker engine + compose 插件
> sudo usermod -aG docker $USER                    # 然后重新登录（或执行：newgrp docker）
> ```

> **pgvector**：已内置在 `pgvector/pgvector:pg16` 镜像中，无需单独安装。应用启动时会自动启用该扩展（`CREATE EXTENSION IF NOT EXISTS vector`）。

方案 B — 原生 PostgreSQL 16（macOS Homebrew）

```
brew install postgresql@16 pgvector   # pgvector 用于向量嵌入，必需
brew services start postgresql@16
# 端口 :5432；创建角色 + 数据库（或让 seed_all.py 自动创建——当应用用户
# 能以超级用户连接时它会自动建库）：
psql postgres -c "CREATE ROLE langconfig LOGIN PASSWORD 'langconfig_dev' CREATEDB;"
psql postgres -c "CREATE DATABASE langconfig OWNER langconfig;"
```

方案 C — 原生 PostgreSQL 16（Ubuntu / WSL2）

Ubuntu 24.04 —— 发行版仓库自带 PostgreSQL 16 + pgvector：

```
sudo apt install -y postgresql-16 postgresql-16-pgvector   # pgvector 用于向量嵌入
sudo service postgresql start
sudo -u postgres psql -c "CREATE ROLE langconfig LOGIN PASSWORD 'langconfig_dev'
    CREATEDB;"
sudo -u postgres psql -c "CREATE DATABASE langconfig OWNER langconfig;"
```


2 / 12

Ubuntu 22.04 —— 自带 PostgreSQL 14 且没有 pgvector 包，需要先添加官方 PGDG apt 仓库（提供 Postgres 16 + `postgresql-16-pgvector`），再执行与上面 相同的命令：

```
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc --fail
        https://www.postgresql.org/media/keys/ACCC4CF8.asc
sudo sh -c 'echo "deb [signed-by=/usr/share/postgresql-
        common/pgdg/apt.postgresql.org.asc] https://apt.postgresql.org/pub/repos/apt
        $(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'
sudo apt update
sudo apt install -y postgresql-16 postgresql-16-pgvector
sudo service postgresql start
sudo -u postgres psql -c "CREATE ROLE langconfig LOGIN PASSWORD 'langconfig_dev'
        CREATEDB;"
sudo -u postgres psql -c "CREATE DATABASE langconfig OWNER langconfig;"
```

方案 D — 原生 PostgreSQL 16（RHEL / Rocky / Alma / CentOS Stream / Fedora，dnf）

```
# RHEL 9 / Rocky / Alma / CentOS Stream 9（RHEL 8 系列改用 EL-8）：
sudo dnf install -y https://download.postgresql.org/pub/repos/yum/reporpms/EL-9-
        x86_64/pgdg-redhat-repo-latest.noarch.rpm
sudo dnf -qy module disable postgresql
sudo dnf install -y postgresql16-server postgresql16 pgvector_16   # pgvector_16 =
        PG16 的 pgvector
sudo /usr/pgsql-16/bin/postgresql-16-setup initdb
sudo systemctl enable --now postgresql-16
# 端口 :5432；创建角色 + 数据库（或让 seed_all.py 自动创建）：
sudo -u postgres psql -c "CREATE ROLE langconfig LOGIN PASSWORD 'langconfig_dev'
        CREATEDB;"
sudo -u postgres psql -c "CREATE DATABASE langconfig OWNER langconfig;"
```

> Fedora 自带 PostgreSQL 16 + `pgvector`： `sudo dnf install -y postgresql16-server postgresql16 pgvector`（跳过 PGDG 仓库 步骤；initdb 与 systemctl 步骤相同）。

Neo4j —— 可选，用于 Cypher 智能体 / Graph Studio

`neo4j_agent`（Graph Studio）通过 **Bolt** 协议查询 Neo4j 图数据库。如果团队只用 SQL 智能体，可以跳过本节。

方式 1 —— Docker（本地，免费）：

```
docker run -d --name langconfig-neo4j \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/password \
  neo4j:5-community
# Web 界面：http://localhost:7474   ·   Bolt：bolt://localhost:7687
```


3 / 12

方式 2 —— Neo4j Aura（云端，免费额度）： 在 Aura 控制台创建实例，使用它提供的 Bolt URI 和凭据。

.env —— NEO4J_* 配置块（`env_example` 已含这些值；把它们加进你的 `.env`）：

```
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=password
NEO4J_DATABASE=neo4j
NEO4J_ENCRYPTED=false
NEO4J_URL=http://localhost:7474   # HTTP 端点（文档索引相关工具）
```

> **Aura**：使用 Aura 提供的 URI，并把 `NEO4J_ENCRYPTED=true`。

加载示例图（智能体的 e2e 测试使用 Movies 图——在 Web 界面执行 `:play movies` 可加载完整数据集；下面是最小种子，用于快速验证）：

```
docker exec langconfig-neo4j cypher-shell -u neo4j -p password <<'CYPHER'
CREATE (m1:Movie {title:'The Matrix', released:1999}),
       (m2:Movie {title:'The Matrix Reloaded', released:2003}),
       (k:Person {name:'Keanu Reeves', born:1964}),
       (l:Person {name:'Lana Wachowski', born:1965}),
       (k)-[:ACTED_IN {roles:['Neo']}]->(m1),
       (k)-[:ACTED_IN {roles:['Neo']}]->(m2),
       (l)-[:DIRECTED]->(m1), (l)-[:DIRECTED]->(m2)
CYPHER
```

验证：

```
cd backend && uv run python langgraph/test_neo4j_agent.py unit
# 然后打开 Graph Studio → neo4j 智能体 → 数据库下拉框会显示图的标签
```

团队种子 —— 企业 HR 图谱（随本包分发：`backend/scripts/seeds/neo4j-hr/neo4j.dump`，约 19 MB——`neo4j_agent` 演示用的 知识图谱：Person / Skill / Job / Domain / WorkType + KNOWS / BUILT / LED …）。 把它恢复到全新的 Neo4j 容器：

4 / 12

```
# 1. 启动 Neo4j 容器（与源版本一致 —— 企业版镜像）：
docker run -d --name langconfig-neo4j-hr \
  -p 17475:7474 -p 7688:7687 \
  -e NEO4J_AUTH=neo4j/password \
  -e NEO4J_ACCEPT_LICENSE_AGREEMENT=yes \
  -v neo4j-hr-data:/data \
  neo4j:2025.10.1-enterprise

# 2. 停止 → 用 dump 覆盖默认数据库 → 重新启动：
docker stop langconfig-neo4j-hr
docker run --rm -e NEO4J_ACCEPT_LICENSE_AGREEMENT=yes \
  -v neo4j-hr-data:/data -v "$(pwd)/backend/scripts/seeds/neo4j-hr":/dumps \
  neo4j:2025.10.1-enterprise \
  neo4j-admin database load neo4j --from-path=/dumps --overwrite-destination=true
docker start langconfig-neo4j-hr

# 3. 让 LangConfig 指向它并验证：
#   NEO4J_URI=bolt://localhost:7688    （其余 NEO4J_* 配置不变）
cd backend && uv run python langgraph/test_neo4j_agent.py unit
#   → Graph Studio 会显示 Person/Skill/Job/Domain/WorkType 标签
```

> **版本兼容性（重要）：** Neo4j 的 dump 只能恢复到**相同或更新**版本的服务器。该 HR dump 来自 **2025.10.1**，因此容器必须使用 `neo4j:2025.10.1-enterprise`（或更新版本）。较旧的 `neo4j:5.x`（5.26）镜像会拒绝它，报错：*"The loaded database is of a newer version than the current binaries. Downgrade is not supported."*

> 要做 SQL 与 Cypher 的对照演示，可以用同样的方式把 Northwind 图（或 Neo4j 欺诈演示数据集）加载进来——使用你自己的 Cypher 文件或 Aura 备份。

> 角色需要 `CREATEDB` 权限，因为 `seed_all.py` 第 1 步会在数据库缺失时创建它， 第 10 步会为 Graph Studio 创建 `chinook` 示例数据库。

---

2. 安装 uv（Python 3.13 管理器）

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
# 然后重启你的 shell 或执行：  source ~/.local/bin/env    （路径会自动添加）

uv --version          # 验证
```

需要 Python **3.13+**（仓库要求 `>=3.13`）。如果你的系统 Python 版本较旧， uv 可以自动拉取：

5 / 12

```
uv python install 3.13
```

3. 解压安装包

```
unzip langconfig.zip -d langconfig
cd langconfig
ls          # AGENTS.md  backend/  docker-compose.yml  docs/  package.json  src/
        start.sh  scripts/ ...
```

4. 同步 Python 依赖（`uv sync`）

项目的 Python 清单是 **`backend/pyproject.toml`**（FastAPI、LangChain 1.x、LangGraph、deepagents 0.7.1、pgvector、Playwright 等）。`uv sync` 读取它并创建 `backend/.venv`。

```
cd backend
uv sync                 # 创建 .venv/ 并安装所有锁定依赖
uv run python -c "import fastapi, langgraph, deepagents; print('deps OK')"
```

> `backend/requirements.txt` 也存在，作为 pip 备选方案（`pip install -r requirements.txt`），但 **`uv sync` 是标准路径**。前端依赖是独立的：`cd .. && npm install`（仅 UI 需要；纯后端 API 不需要）。

Node.js —— 前端 UI 必需

UI 需要 Node **20.19+**（Vite 7 要求）。通过 nvm 安装：

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
export NVM_DIR="$HOME/.nvm"; [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
nvm install 22
node --version && npm --version    # 验证（node ≥ 20.19，npm ≥ 10）
```

然后在仓库根目录执行 `npm install`（一次性）——前端启动（第 9 步）前需要它。

5. 配置 `.env`（含工作区）——必须在种子之前，按所有者划分

`.env` 是**引导默认值**（首次启动时读取一次）。用户可调的运行时设置和所有 API Key 都在数据库中，可通过 Settings 面板修改。三个分区，按顺序如下：

6 / 12

```
# backend/../.env   (仓库根目录)
# 从仓库里的 env_example 模板开始 (已提交), 再修改以下三个分区:

# ═══ OPERATOR-ONLY (基础设施 — 不要让用户改) ═══
POSTGRES_HOST=localhost
POSTGRES_PORT=5433          # docker; 原生用 5432
POSTGRES_DB=langconfig
POSTGRES_USER=langconfig
POSTGRES_PASSWORD=langconfig_dev
API_KEY_ENCRYPTION_KEY=用 `openssl rand -hex 32` 的输出替换
ENVIRONMENT=development
TEST_DATABASE_URL=postgresql+asyncpg://langconfig:langconfig_dev@localhost:5433/langconfig_test
SQL_POSTGRES_DB=chinook
GOOGLE_REDIRECT_URI=http://localhost:8765/api/auth/google/callback
OLLAMA_BASE_URL=http://localhost:11434
SQL_ECHO=false
RATE_LIMIT_CLEANUP_INTERVAL=300
# Neo4j (可选 —— Cypher 智能体 / Graph Studio; 见第 1 步 "Neo4j"):
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=password
NEO4J_DATABASE=neo4j
NEO4J_ENCRYPTED=false

# ═══ USER-ADJUSTABLE (引导默认值 — UI 启动时会覆盖) ═══
# 这些值会在首次启动时种入 'runtime' 命名空间。改这里等于改默认值;
# 团队一旦在 Settings 面板里设了值, UI 始终胜出。
LANGCONFIG_WORKSPACE_ROOT=backend/.workspace_data
STORAGE_MODE=db
OLLAMA_EMBED_MODEL=nomic-embed-text:latest
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langconfig
BACKUP_DIR=./backups
DB_POOL_SIZE=10
DB_MAX_OVERFLOW=20
ANALYSIS_RECURSION_LIMIT=1000
ANALYSIS_DASHBOARD_TIMEOUT=600
ANALYSIS_CHAT_TIMEOUT=120
MAX_EXECUTION_HISTORY_PER_WORKFLOW=100
EXECUTION_HISTORY_RETENTION_DAYS=90
AUTO_CLEANUP_EXECUTION_HISTORY=true
RATE_LIMIT_ENABLED=true
RATE_LIMIT_REQUESTS_PER_MINUTE=1000
PERFORMANCE_MONITORING_ENABLED=true
SLOW_REQUEST_THRESHOLD_MS=1000
LOG_SLOW_QUERIES=true
SLOW_QUERY_THRESHOLD=1.0
LOG_ALL_REQUESTS=false

# ═══ API KEYS — 不要放进 .env ═══
# 所有密钥都在加密的 models.api_keys 设置文档里
#  (Settings → Models, 或首次启动向导)。上方的 API_KEY_ENCRYPTION_KEY
# 是静态加密这些密钥的 Fernet 主密钥。
```


7 / 12

工作区参数——每个智能体的文件夹到底在哪

团队真正关心的目录是**每个智能体的输出目录**（智能体往这里写文件）。它在 **每次聊天会话时**解析（`WorkspaceManager.resolve_agent_workspace`），优先级如下：

1. **用户自定义**——`deep_agent_templates.workspace_path`（每个智能体单独设置）； 绝对路径直接使用，相对路径锚定到 vault 根目录。
2. **Vault 作用域**（会话绑定到项目，`chat_sessions.project_id`）——`{workspace_base}/{vault_key}/agents/{agent_key}/`，其中 `vault_key` 是项目 名的 slug，`agent_key` 是智能体名的 slug。
3. **旧版默认**——会话没有项目绑定时，使用 `{workspace_base}/agents/agent_{id}/`。

身份信息（system prompt + AGENTS.md/CLAUDE.md）会在**每次聊天时用会话解析出的 根目录重新烘焙**（docs/system-architecture.md §16.10），因此绑定到项目的智能体 会报告 vault 作用域路径。种子和回填脚本都是与项目无关的，所以它们烘焙的是 旧版默认值，除非你设置了 `workspace_path`。

团队可以通过三种方式调整布局：

1. **DB 里每个智能体的 `workspace_path`**（推荐覆盖方式）。把 `deep_agent_templates.workspace_path` 设为绝对路径（或相对于 Settings 文档 基础目录的相对路径）。在 Settings 面板对每个智能体单独设置。
2. **回填脚本批量更新**（`scripts/backfill_agent_identity.py`）：在给定根目录下 重新烘焙每个智能体的身份 + 文档，带 `agent_{id}/` 子目录——这是同时调整所有 智能体文件系统布局的唯一方式（无项目绑定 → 该根目录下的旧版布局）。
3. **`.env` 里的 `LANGCONFIG_WORKSPACE_ROOT` 不是智能体根目录**。这个变量只在 `services/agents_md_service.py` 里被读取，用来定位项目级的 `AGENTS.md` 目录 （在 `<root>/{project_id}/` 下）。它**不**用于智能体级 workspace 解析、 种子或回填。

简单说：回答"是否需要在 `.env` 里设 `LANGCONFIG_WORKSPACE_ROOT` 然后跑 seed_all.py？"这个问题的答案是**不需要**——种子总是用默认值，除非你显式设置每个智能体的 `workspace_path` 列。运行时，绑定到项目的会话会自动解析为 vault 作用域目录（`{vault_key}/agents/{agent_key}/`，§16.10）；如需覆盖，设置 `workspace_path` 或重新运行回填脚本。

---

8 / 12

6. 配置 LLM API Key（在数据库里，不在 `.env` 里）

所有 API Key 都在数据库里（`models.api_keys` 设置文档），用 `.env` 上面的 `API_KEY_ENCRYPTION_KEY` 静态加密存储。`.env` 不再是 API Key 的来源——Settings 面板（或首次启动向导）是唯一的编辑路径。

解析顺序（`config.py::get_api_key_from_db`）：

1. **PostgreSQL** — `models.api_keys` 字段（通过 Settings → Models 设置）
2. `.env` — 仅作为首次启动的引导默认值（已不推荐）

团队安装建议用 Settings UI：

1. **首次启动向导**——启动后打开 `http://localhost:1421`，Setup 向导会提示输入 API Key + 工作区路径。
2. **启动后 Settings → Models**——添加各提供商 Key（OpenAI / Anthropic / DeepSeek / Google / MiniMax / Tavily 等）。

> 仓库的 `env_example` 不含任何 API Key（遵循上方 "API KEYS — 不要放进 `.env`" 分区）。如果是迁移旧的 `.env`，跑一次性脚本把 Key 移进数据库：`uv run python scripts/import_api_keys_to_db.py`（幂等，支持 `--dry-run`）。
>
> 你需要哪些 Key 取决于智能体使用的模型。种子智能体默认使用 Anthropic（`claude-sonnet-4-5`）/ DeepSeek 模型——至少添加与你默认模型匹配的提供商 Key，例如 `ANTHROPIC_API_KEY` 或 `DEEPSEEK_API_KEY`（Settings → Models）。

7. 运行数据库种子

在 `backend/` 目录下执行：

```
cd backend
WORKSPACE_ROOT=/Users/<you>/langconfig/backend/outputs \
  uv run python scripts/seeds/seed_all.py
```

它会做什么（10 步，全部幂等）：

步骤	操作
1	创建 Postgres 角色 + 数据库（若缺失）
2	创建所有表 + pgvector 扩展
3	alembic stamp head

步骤	操作
4–6	种子设置文档、28 个模型、6 个搜索提供商
7	恢复 58 个工具
8	恢复 44 个深度智能体（与机器无关的 dump）
8.5	**身份回填**——将每个智能体的真实 id + `<WORKSPACE_ROOT>/agents/agent_{id}` + AGENTS.md/CLAUDE.md 烘焙进 `system_prompt` + `agent_docs`
9	从 `backend/skills/` 同步 19 个技能
10	创建 `chinook` 示例数据库（尽力而为）

验证结果（无论你选择什么工作区根目录都能生效）：

```
uv run python -c "
from db.database import SessionLocal
from models.deep_agent import DeepAgentTemplate
db = SessionLocal()
a = db.query(DeepAgentTemplate).filter_by(id=1).first()
sp = (a.config or {}).get('system_prompt','')
print('identity:', '## Identity' in sp, '| agent_id baked:', 'Agent ID**: 1' in sp,
        '| no placeholder:', 'WORKSPACE_BASE' not in sp and '{{' not in sp)
db.close()
"
# 预期输出：identity: True | agent_id baked: True | no placeholder: True
```

> 之后要更改工作区，只需重新运行回填：`uv run python scripts/backfill_agent_identity.py --workspace-root <新路径>`

---

8. 启动前健康检查

`scripts/health_check.sh` 会在启动完整堆栈**之前**验证 Postgres、API、 数据库连通性和种子数据：

```
bash scripts/health_check.sh                # 默认后端 :8765
BACKEND_PORT=8799 bash scripts/health_check.sh   # 自定义端口
```

预期输出（全部 ✅）：

```
── [1/4] PostgreSQL ──────────────  ✅ PostgreSQL accepting connections on :5433
── [2/4] Backend API ─────────────  ✅ Backend reachable on :8765
── [3/4] Detailed health ─────────   overall: healthy | database: healthy
── [4/4] Seeded data ─────────────  ✅ deep agents in DB: 44
✅ ALL CHECKS PASSED — ready to run:  bash start.sh
```

如果失败，它会为每一步打印修复方法（启动 Postgres → 种子 → 重试）。

10 / 12

> 注意：该脚本需要后端**已经运行**才能检查第 2–4 步。 首次启动：先启动一次后端（`bash start.sh --be`），运行检查，再执行 `bash start.sh` 启动完整堆栈。

9. 启动完整堆栈

```
bash start.sh
```

进程	地址	说明
后端（FastAPI，2 个 worker）	`http://localhost:8765`	Swagger 文档在 `/docs`
前端（Vite/React）	`http://localhost:1421`	VS Code 风格 IDE 外壳
PostgreSQL	`:5433`（docker）/ `:5432`（原生）	

常用变体：

```
bash start.sh --be    # 仅后端（前端保持运行）
bash start.sh --fe    # 仅前端
```

首次登录：Setup 向导会询问你的 API Key + 工作区路径——这里输入的工作区应与 第 5 步的 `WORKSPACE_ROOT` 一致，这样烘焙进智能体的身份指向运行时使用的 同一个文件夹。

故障排查

症状	修复
`psycopg ... connection refused :5432/:5433`	Postgres 未启动 → 第 1 步（`docker compose up -d postgres` / `brew services start postgresql@16` / `sudo systemctl enable --now postgresql-16`）
`extension "vector" is not available` / 嵌入报错	操作系统未安装 pgvector → 按第 1 步对应方案安装（Docker 已内置；原生：`brew install pgvector`、`apt install postgresql-16-pgvector`、`dnf install pgvector_16`）
`npm: command not found` / 前端无法启动	未安装 Node → 用 nvm 安装（第 4 步 → "Node.js —— 前端 UI 必需"），然后 `npm install`
`neo4j ... could not connect` / 认证失败 / Graph Studio 无标签	Neo4j 未运行或凭据错误 → 启动容器（第 1 步 → "Neo4j"），检查 `.env` 中的 `NEO4J_*` 配置块（Aura：设 `NEO4J_ENCRYPTED=true` 并使用 Aura URI），然后 `uv run python langgraph/test_neo4j_agent.py unit`
种子第 1 步打印手动 CREATE ROLE 说明	应用用户无法以超级用户连接 → 运行它打印的两条 `psql` 命令，然后重新运行 `seed_all.py`

症状	修复
健康检查后 `deep agents: 0`	运行第 7 步；确认 `WORKSPACE_ROOT` 已导出
智能体 prompt 中出现 `WORKSPACE_BASE/` 或 `{{...}}`	回填未运行 → `uv run python scripts/backfill_agent_identity.py --workspace-root <根目录>`
前端 :1421 空白页	`npm install` 然后 `bash start.sh --fe`
API Key 不生效	在 Settings → Models 中添加（数据库，加密）；确认 `.env` 中已设置 `API_KEY_ENCRYPTION_KEY`

12 / 12